

2025 中国研究生创“芯”大赛·EDA 精英挑战赛作品设计报告

赛题一：支持重新组网的多 FPGA 系统布线算法设计

参赛 ID：EDA250103

二〇二五年十一月

目录

一. 问题描述与分析.....	3
二. 核心思想与创新点.....	5
三. 基于模拟退火机制的自适应邻域搜索算法设计.....	7
3.1 总算法框架.....	7
3.2 基于动态贪心的路径初始化.....	8
3.3 邻域 1——基于束搜索的单路径重构.....	9
3.4 邻域 2——随机贪心的单路径重构.....	11
3.5 邻域 3——通道增加与再分配.....	12
3.6 邻域 4——全局破坏与重构.....	13
3.7 基于多臂老虎机决策思想的邻域自适应选择策略.....	14
3.8 重启动机制.....	16
3.9 模拟退火.....	16
四. 实验测试.....	18
4.1 实验结果分析.....	18
4.2 收敛性分析.....	18
4.3 消融实验.....	19
4.4 自适应邻域选择策略有效性.....	20
五. 总结与展望.....	22
5.1 总结.....	22
5.2 未来展望.....	22
参考文献.....	23

一. 问题描述与分析

本赛题要求在给定多 FPGA 系统的初始物理拓扑结构以及逻辑 net 映射的基础上, 为所有跨 FPGA 的 nets 分配路径, 并在满足 TDM 比率约束、通道资源约束、通道变动量约束等限制的前提下, 最小化所有 nets 中的最大路径延迟。

从单个 net 的路由来看, 该问题本质上是一个多接收端的最小延迟路径问题。算法需要为源端到每一个接收端规划一条有效路径, 并最小化这些路径中延迟的最大值。但是, 从整体来看, 路径选择不仅影响单个 net 的通信性能, 也会由于跨通道 net 数量的变化而影响全局 TDM 比率, 从而进一步改变其他 nets 的延迟。这使得布线过程呈现出显著的全局耦合性, 导致优化难度大幅提升。

与传统的布线问题相比, 本赛题有以下特点: 其一, 相同 net 内部同一通道只计一次并要求 TDM 比率取 8 的整数倍的情况下, 增加跨单个通道的 net 数量的边际代价呈现阶梯型而非线性。这一特性导致路径延迟对通道占用并非连续敏感, 而是表现出明显的非线性跃迁, 使局部微小改动可能引发延迟突变, 从而加剧了搜索空间的不规则性。其二, 目标函数并非总延迟或总功耗, 而是所有 nets 中路径延迟的最大值, 算法因此必须兼顾整体平均性能和最坏路径性能的平衡, 更多关注全局瓶颈而非局部累计收益。其三, 物理拓扑结构并非完全固定, 允许对通道数量进行有限度的调整, 这使问题从固定图上的路由单项优化转化为拓扑结构与路由的协同优化, 变化拓扑结构不仅改变路径可达性, 还直接影响系统整体的 TDM 比率分布, 使路由优化与拓扑优化形成强耦合关系, 增加了解搜索维度与全局协调难度。

在单 FPGA 布线资源协调方面, McMurchie 等^[1]开创性地提出了 PathFinder 算法, 利用“资源协商”机制, 通过反复布线与拥塞惩罚系数调整, 引导布线逐步趋于合法解, 对后续算法有重要的启发意义。

当前多 FPGA 布线问题的研究面临诸多挑战, 主要包括跨 FPGA 信号数量激增所带来的资源瓶颈、TDM 技术引入的延迟优化难题、以及系统拓扑与路径路由协同设计的复杂性。针对这些问题, 已有研究从不同角度展开探索。Turki 等提出了一种迭代式的跨 FPGA 信号布线算法, 专注于多 FPGA 原型验证平台中跨芯片连接的路径规划问题。通过采用路径局部优化与冲突修复相结合的策略, 在保证布线资源限制的基础上逐步改进跨通道连接, 提升了解可行性与布线成功

率^[2]。Tung 等^[3]则将布线拓扑选择与 TDM 调度纳入统一优化框架，提出了一种基于 Lagrangian 松弛的迭代联合优化算法，通过建立整型线性规划模型，并采用子问题分解的方式，实现对路径选择与 TDM 比率的协同最小化。Zou 等^[4]面向大型逻辑验证任务中的系统级 FPGA 布线问题，提出了一种基于 TDM 的路径构造与同步调度方法，可以在考虑跨模块信号同步、TDM 时间槽分配与拓扑可行性，满足时序约束的同时优化整体通信开销。Khalid 等^[5]则从架构角度出发，设计了一种新颖的多 FPGA 路由结构，包括可配置逻辑块之间的互连方式与通道资源的布局方案。从结构设计方面提出了优化策略。

综上，现有工作多分别从路径级、TDM 调度级与结构级切入。而基于上述对问题结构的理解，本赛题的布线优化需要在“局部路径调整”，“全局拥塞控制”，以及“有限度的拓扑重构”三者之间取得平衡，需要综合考量以上方面实现协同调度的优化框架。此外，考虑到实际场景中往往存在数量级可达百万的信号需求，路径搜索的计算复杂度极高，且 TDM 成本高度依赖系统负载，精确求解的代价难以接受。因此，在该赛题背景下，本研究不使用传统的精确整数规划方法，而是采用结合启发式路径搜索、局部增量更新与有限拓扑重构的高效近似算法，并针对该问题特征进行搜索机制的设计。

二. 核心思想与创新点

基于第一章对多 FPGA 系统级布线问题的分析可知, 该赛题本质上属于具有 NP-hard 特性的组合优化问题。由于跨 FPGA nets 的路由决策存在强耦合效应, 传统的精确算法和商业求解器在面对大规模实例时难以在合理时间内获得可行解或高质量解, 因此无法满足实际系统设计的需求。该问题同时具备多接收端路径规划、类斯坦纳树结构构造以及全局 TDM 比率反馈调节等特征, 使得候选解空间呈指数级增长; 再加上通道容量约束、路径延迟约束以及通道变动量约束等多类限制共同作用, 使得搜索空间高度离散且崎岖, 算法极易陷入局部最优, 从而进一步增加了求解难度。

针对上述挑战, 本研究构建了一套结合多层次扰动机制与自适应控制策略的启发式优化框架, 并在此基础上定制化设计了一个基于模拟退火思想的自适应邻域搜索算法, 用于在全局探索能力与局部精细优化之间取得平衡。该算法能够在严格约束条件下有效跳出局部最优, 逐步逼近全局高质量解。该算法核心思想和创新点介绍如下:

(1) 多层次协同多邻域结构设计

启发式算法的性能在很大程度上取决于邻域结构的设计, 而邻域的多样性与层次性能够显著提升探索深度与跳出局部最优的能力。在求解本赛题时, 单一的局部移动往往不足以突破复杂约束所形成的搜索瓶颈。鉴于本问题同时存在单条路径局部阻塞、通道容量紧张以及多 net 全局交互等多维度困难, 有必要构建覆盖不同尺度的多邻域体系, 以支持算法在局部精修与大范围扰动之间灵活切换。

基于上述考虑, 本算法设计了四类具有协同作用的邻域结构: 基于束搜索的路径重构邻域 N_1 ; 随机贪心的最短路径重构邻域 N_2 ; 通道增加与再分配邻域 N_3 ; 全局破坏与重构扰动邻域 N_4 。这些邻域覆盖了从单路径微调到跨 net 的大规模结构重构等多层次操作, 为搜索过程提供丰富而有效的解空间探索能力, 是本算法的核心设计亮点之一。

(2) 基于模拟退火的搜索平衡机制

由于本赛题解空间高度离散且存在大量局部最优, 过度依赖贪心更新会使搜索在早期迅速收缩到次优区域。为提升算法在不同阶段的搜索质量, 本研究采用模拟退火 (Simulated Annealing, SA)^[6, 7]作为邻域变换后的解接受机制。在高温

阶段, SA 允许以较高概率接受由邻域操作产生的劣解, 帮助搜索跨越路径拥塞、通道容量限制等局部障碍, 从而扩大探索范围。随着温度逐步降低, 接受概率自然收敛, 搜索策略由全局探索平稳过渡到局部精细优化, 实现启发式算法强化搜索 (Intensification) 与扩散搜索 (Diversification) 的动态平衡。该机制有效抑制了早熟收敛风险, 并保证算法具备良好的收敛性。

(3) 基于多臂老虎机思想的邻域自适应调控策略

在本研究所设计的自适应邻域搜索框架中, 不同邻域在搜索的不同阶段往往具有差异化的效果。为了提升邻域选择的效率, 本研究引入多臂老虎机 (Multi-Armed Bandit, MAB)^[8] 思想, 对各邻域的尝试次数与成功次数进行动态记录, 并利用 Softmax 评分结合轮盘赌机制, 对邻域选择概率进行自适应更新。该策略能够自动偏向近期表现更优的邻域, 同时保留对其他邻域的必要探索, 在利用与探索之间实现平衡, 使整个搜索过程能够根据问题状态的变化自发调整行为。

(4) 面向大规模搜索停滞的重启动机制

考虑到本赛题在大规模布线实例中, 局部区域的搜索停滞会显著增加时间成本。为此, 算法设计了重启动机制: 当连续迭代无改进的次数达到设定阈值时, 算法会随机扰动 net 的处理顺序并重新生成初始解, 再在新的解域中继续搜索。该机制能够有效避免搜索在劣质局部区域反复震荡, 进一步增强算法跳出局部最优的能力。

三. 基于模拟退火机制的自适应邻域搜索算法设计

3.1 总算法框架

基于对多 FPGA 系统布线问题的分析,本研究设计了基于模拟退火机制的自适应邻域搜索算法对该问题进行求解。算法流程图如图 3-1 所示。

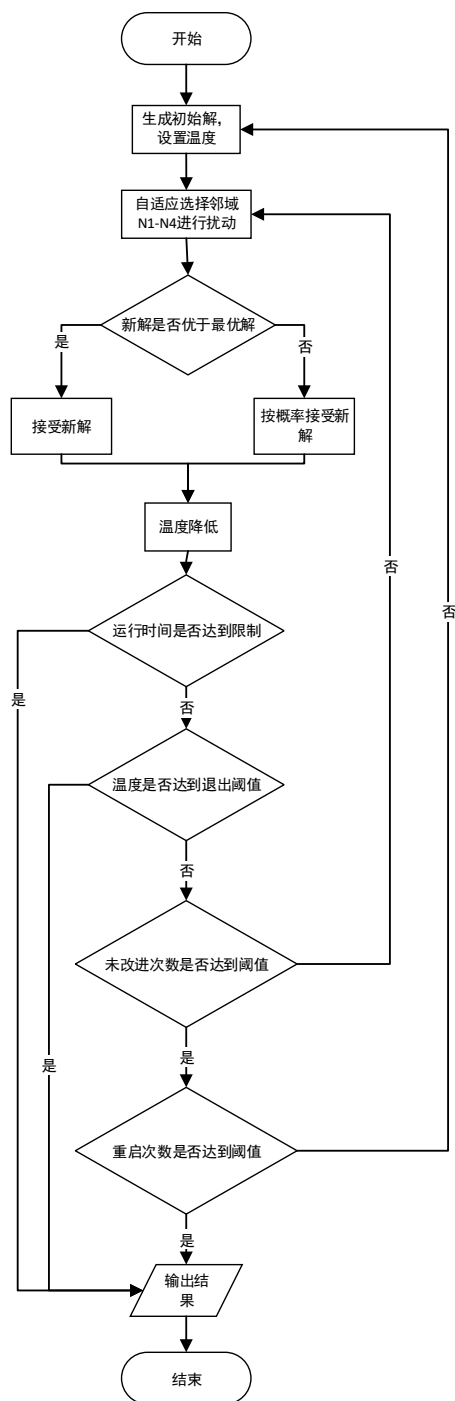


图3-1 算法流程图

该算法以生成一份可行初始解并设定温度为起点,初始解通过动态贪心策略

构造, 确保在满足资源约束的前提下提供合理起点。随后, 算法进入主循环, 在运行时间未达上限且温度未降至退出阈值前不断执行“邻域扰动—解接受判定—重启检测”三个阶段, 以逐步提升解的质量。

在每轮迭代中, 算法首先利用基于多臂老虎机模型的自适应机制选择一个邻域 (N_1 - N_4) 对当前解进行扰动。若扰动后产生的候选解质量优于最优解, 则直接接受该候选解; 否则, 根据模拟退火接受准则, 以一定概率接受该解。

接受新解后, 系统会同步更新邻域的成功记录, 用于后续自适应选择。同时, 若当前解优于迄今为止的最优解, 算法将更新全局最优解并重置停滞计数器。若连续多轮未出现更优解, 系统则触发重启机制, 随机打乱 `net` 顺序, 重新构造初始解, 并重置温度, 从而跳转至全新解域继续探索。最终, 算法在温度降至预设下限或运行时间达到限制时终止, 并返回记录的最优解。

3.2 基于动态贪心的路径初始化

初始化阶段, 为高效处理多接收端 `nets`, 算法采用了一种基于 Dijkstra^[9] 的最短路径树策略。对于每条 `net`, 算法不会对其每个接收端重复运行最短路搜索, 而是仅从源端 FPGA 出发运行一次 Dijkstra, 构建一棵到所有相关 FPGA 节点的最短路径树。`net` 的所有接收端 FPGA 都可以从同一棵树中直接回溯得到对应路径, 从而显著降低计算成本。同时, 考虑到路径构建顺序对初始解质量的影响, 算法会随机选择各 `net` 的路径规划顺序, 使得初始解保持一定的随机性。

在最短路的边代价设置过程中, 算法不仅考虑了跨 FPGA 带来的固定延迟, 也综合评估了当前通道的负载情况。具体而言, 对于每条 FPGA 之间的通道, 算法会根据其当前承载的跨 FPGA 数据量设置相应的搜索代价值: 负载较高的边代价较大, 而负载较轻、通道资源更充裕的边代价较小。通过这种方式构建的最短路径树, 使初始化得到的路径更倾向于避开拥塞区域, 从而在起点阶段就形成较为均匀的全局负载分布。

对于源端与接收端位于同一 FPGA 的情况, 算法将直接判定该连接无需跨 FPGA 通道, 路径设为空, 不占用任何外部通信资源。

在最短路径树生成之后, 初始化阶段通过回溯每个接收端的父节点关系, 得到具体的跨 FPGA 路径, 并将这些路径逐段加入系统的通道使用统计结构中。随后, 根据路径中跨 FPGA 段的数量与通道的当前负载, 计算各条路径的延迟值,

并进一步得到整条 net 的最坏路径延迟。通过对所有 nets 重复上述过程，整个系统的初始跨通道使用情况、初始延迟分布以及后续拓扑可调整范围等信息便得以完整建立。

3.3 邻域 1——基于束搜索的单路径重构

为达到搜索质量与搜索效率的权衡，以不消耗过多资源地维持解的多样性，使搜索不过早地陷入局部最优，本研究设计了基于束搜索的单路径重构邻域。

束搜索（Beam Search, BS）^[10, 11]是一种广泛应用于序列生成模型（如机器翻译、语音识别、文本生成等）的启发式搜索算法。它在广度优先搜索（Breadth-First Search, BFS）^[12]的基础上引入了高效的剪枝策略。已有较多研究将束搜索应用到启发式优化中。Nazari 等^[13]人将束搜索引入到物流路径规划问题中，通过在每一步保留多个候选决策，显著提升了最终解的质量，并在效率与计算开销之间取得了良好平衡。Choo 等^[14]则提出 SGBS（Simulation-Guided Beam Search），将束搜索方法与蒙特卡洛模拟结合，应用于神经网络优化框架，在旅行商问题与带容量约束的路径规划中表现出优于传统神经启发式方法的效果。

束搜索的过程示意如图 3-2 所示：与 BFS 会保留所有路径不同，束搜索在搜索的每一层（或每一步），都会保留一个固定宽度 K 的“束”（Beam），即只保留 K 个当前被评估为“最优”的候选路径。在下一层扩展时，算法只从这 K 条路径出发，并从它们生成的所有后继路径中，再次筛选出新的 K 条最优路径。相比于贪心搜索，束搜索通过保留多个候选序列，能够有效避免局部最优解，找到更好的全局解。如图 3-2 中，束搜索在每层都保留 $K=2$ 个当前“最优”的候选路径对应的节点：假设从某条路线的起点出发，其下一步的候选路径中最优的两个节点为 A 和 C ，随后，算法分别从 A 和 C 出发，以同样的方法选出下一层中最优的两个候选节点 B 和 E ，由此逐层拓展，逐步生成新的节点序列，直到达到终点，最终得到 K 条候选路径。

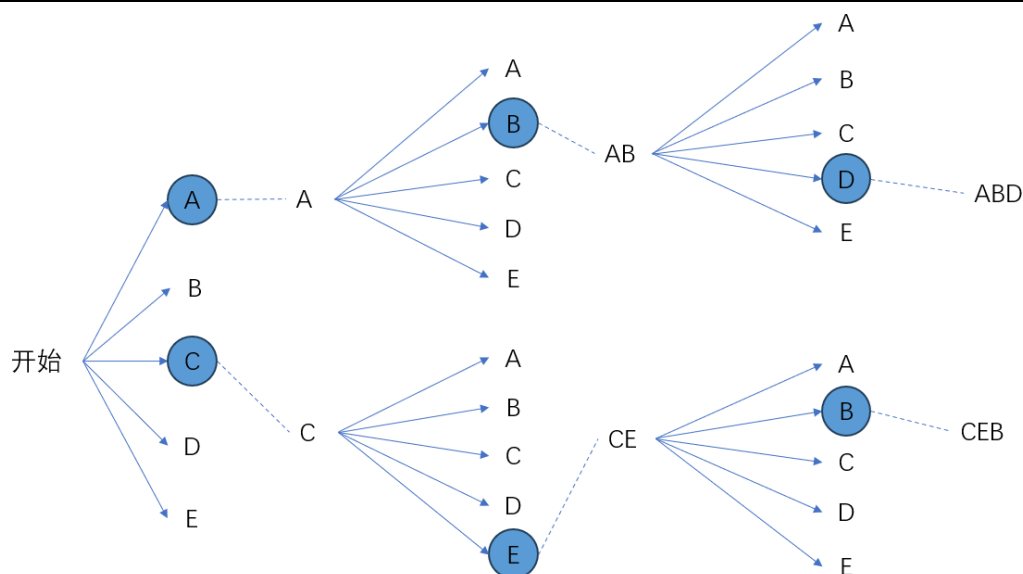


图3-2 束搜索示意图

在定位到当前全局最差的 net 以及其中最差的路径后,邻域 1 会从源端 FPGA 出发执行多层束搜索,用于构建若干条候选替代路径。为了在庞大的 FPGA 间拓扑上高效地引导搜索方向,本研究同时搜索的扩展过程中引入了开放列表和封闭列表的设计。在搜索树的每一次扩展中,算法都会为每条候选路径计算综合延迟评估值,该评估值由当前已走路径的实际累计延迟 G 与从当前节点到目标节点的最短延迟值 H 联合计算,其计算如式(1)所示。

$$F = G + H \quad (1)$$

根据这一评估,算法在评估各个候选分支时,可以综合考虑已经发生的代价并提前预估未来可能付出的最小代价,从而有效提升算法效率。此外,考虑到过深搜索带来的指数级分支增长,为了平衡搜索质量与计算代价,算法在每一层扩展中也采用束搜索的剪枝机制,只保留延迟值最小的若干个候选节点分支,并将其他分支全部剪除,并将搜索过程约束在 5 层之内。如果在达到最大深度后源 FPGA 仍未与目标 FPGA 连接,算法会在当前叶节点处调用一次 Dijkstra 路径连接至终点,保证最终能够生成一组完整的候选路径。

算法在此基础上引入了类似 Boltzmann 选择^[15]的 Softmax 概率机制。首先,邻域 1 会根据路径延迟差异构造一组权重 $\exp(-(d_i - d_0)/\tau)$,其中 τ 由前 20%候选路径的延迟跨度自适应确定,用于控制对延迟差距的敏感程度。随后,算法会根据该分布的累计概率确定一个前缀长度,在该前缀内等概率随机选取一条路径。基于此概率机制,当候选延迟较为接近时,多条路径都有一定被选中的机会,有

利于保持搜索多样性；而当候选间差距明显时，选择会自然偏向延迟更小的路径，从而提升局部收敛效率。

3.4 邻域 2——随机贪心的单路径重构

邻域 2 的设计目标在于为贪心的单路径重构引入一定的随机性。每条路径依然按照最短路贪婪策略选择对应的通道，同时引入随机性，让路径在若干关键节点上有一定概率不选择最短的通道，从而获得更丰富的路径形态和更强的跳出局部最优能力。

具体设计思路为：从起点出发生成一条最短路作为主干，在主干上的每一步，以较大概率贪心地沿着最短路前进。同时，路线也有小概率随机地跳到另一个可达终点的中间节点，并以该节点为起点重新使用 Dijkstra 生成新的最短主干路径，如此反复，直到到达终点。

邻域 2 同样用于重构当前全局最差的 net 以及其中最差的路径，其具体设计如图 3-3 所示：从输入路径的源端 FPGA 出发，调用一次 Dijkstra 算法，生成一条从起点到终点的最短路径，将其视为当前迭代的主干路径。与传统的固定最短路径不同，邻域 2 并不会一次性采用整条路径，而是将其拆解为一系列从当前节点到下一节点的决策步骤。当算法当前处于某个节点 A 时，主干路径已经给出了从 A 到终点的最短路径信息： $A \rightarrow B \rightarrow \dots \rightarrow sink$ 以及其中的下一跳节点 B 。邻域 2 根据预设的概率参数判断是否接受最短路的下一步：以一个较大的概率继续遵循贪心原则接受主干路径，则将 B 选定为路径的下一个节点，并继续从 B 出发进入下一节点决策；同时，也有小概率不再接受主干路径的选择，而是从当前节点 A 的所有邻接节点中，随机选择一个能在满足所有约束条件下可达终点的新节点 D 作为新的下一跳，随后算法会以 D 为新的中间起点，再次调用 Dijkstra 算法生成从 D 到终点的最短路径后缀，并将这一后缀拼接到当前已经确定的前缀路径后面作为新的主干路径。算法将重复上述的决策过程，直到路径节点到达终点，最后生成一条重构后的新路径。

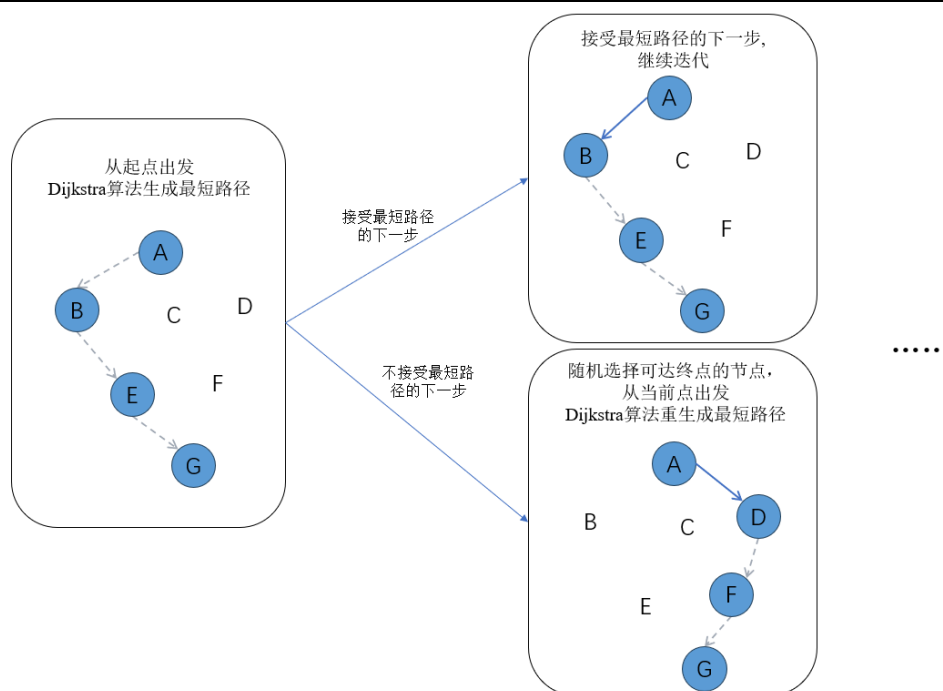


图3-3 概率跳出当前最短路径的路径重构单步示意图

邻域 2 既继承了最短路径构建的优势，又通过逐步概率偏离机制引入了一定的随机性，在效率较高的同时增强了整个算法跳出局部最优、发掘新路径的能力。

3.5 邻域 3——通道增加与再分配

与邻域 1、邻域 2 专注于单条路径的局部重构不同，邻域 3 的作用于系统的物理拓扑结构。其目标是在允许的资源约束范围内对通道数量进行调整，从拓扑结构层面缓解关键链路拥塞，改善全局 TDM 比率，同时为路径调整邻域提供新的潜在路径选择。邻域 3 不仅处理最差路径所涉及的瓶颈边，还会结合全局通道负载信息，对系统中负载最重或潜在阻塞风险较高的区域进行局部拓扑改造，从而提供比路径级优化更高层次的结构改进能力。

邻域 3 的操作由通道新增与通道调整两个阶段构成。在新增阶段，算法根据不同规则选择候选边，并在满足通道资源约束与 TDM 比率约束的前提下，为这些边增加物理通道。为此，邻域 3 中设计了多种增补策略，以兼顾确定性的瓶颈修复与必要的随机探索性。具体规则如下：

(1) 最差路径最差边增加通道

在满足约束的前提下，从当前全局最差路径中选取延迟最高的边，为其增加一条通道，直接削弱其作为全局延迟瓶颈的影响。

(2) 全局高负载边增加通道

在满足约束的前提下，随机选择某条跨 FPGA 数最多的通道，对其增加一条通道以降低 TDM 比率，进而改善路径拥塞情况。

(3) 最差路径内随机边增加通道

在满足约束的前提下，随机选择最差路径中可增通道的一条边，为其增加一条通道，通过对路径局部结构的随机扰动来缓解潜在拥塞。

(4) 全局随机增加通道

在满足约束的前提下，随机抽取 net、其路径及路径中的边，为其增加一条通道，增强算法跳出局部最优的能力。

(5) 当前路径随机增加通道

在满足约束的前提下，随机选择从当前各 net 中各路径中选择一条边，为其增加一条通道，以改善当前路径的 TDM 比率。

(6) 起点与终点之间增加通道

在满足约束的前提下，尝试直接为最差路径的源 FPGA 与目标 FPGA 增加通道，可显著缩短路径跳数并提升路由选择的灵活性。

考虑到仅增加通道可能造成局部拓扑过度偏斜，邻域 3 进一步引入通道调整机制，对新增通道的布局进行再平衡。具体而言，在通道增加阶段结束后，邻域 3 会在满足所有约束与当前解的可行性的前提下，从既未参与任何 net 的实际传输，又不会因回收而破坏 TDM 约束的新增通道中随机移除一定比例的通道，随后按照前文相同的策略再次增加通道，直到达到通道变化数量的规定上限。

通过这种机制，邻域 3 能在不损害现有有效路径的前提下，对新增通道进行安全的再分配，使拓扑结构能够围绕当前解逐步获得更合理的资源布局。通道的调整机制使新增通道获得再次优化的机会，有利于跳出拓扑的局部最优，探索更优的拓扑图网络结构。

3.6 邻域 4——全局破坏与重构

在多 FPGA 布线问题中，TDM 比率直接由各路径的跨通道使用量决定，路径延迟又强依赖于 TDM 比率，因此整个系统呈现出高度耦合的特性。尽管邻域 1 至邻域 3 能在局部范围内对路径结构或通道拓扑进行微调，但一旦整个系统形成某种稳定但低质量的拥塞格局，针对单条路径的邻域便难以凭借小幅度扰动改善这种结构。为重新激活搜索、摆脱既有路径结构对拓扑的约束，本研究在邻域

1 至邻域 3 的执行过程中穿插了全局破坏与重构机制。以成批重构的方式，强制重置部分通道负载，使系统从原有的拥塞中脱离出来，并在新的负载图上重新探索路径结构。

具体而言，邻域 4 首先根据 **nets** 的最坏延迟排序，选取一定比例的延迟最高的 **net**，对其中路径统一删除以破坏当前解。随后，算法会依次调用多终点最短路的 Dijkstra 算法，对被移除的所有 **nets** 进行重新布线。由于负载状态发生了明显改变，这些重构后的路径往往会选择与原解不同的通道组合，从而形成新的全局结构。通过成批删除并重新布线，不仅可以改变被处理 **nets** 自身的路径形态，也可能重塑其他 **nets** 的相对布局，进而为算法提供了跳出局部最优解域的机会，使后续搜索能够在更广的空间中重新展开。

3.7 基于多臂老虎机决策思想的邻域自适应选择策略

为了在复杂的多 FPGA 布线优化过程中合理选择从四个邻域选择每轮迭代所执行的操作，实现局部改进与全局探索之间的平衡，本研究在模拟退火框架中引入了多臂老虎机（MAB）决策思想，构建自适应邻域选择机制。

MAB 源于经典的序贯决策模型，其核心情境是：一个赌徒面对多台带有不同收益分布的老虎机，在不知道每台机器真实收益率的情况下，需要通过反复试验在尝试新拉杆以获取信息与多使用收益更高的拉杆之间取得最优平衡。MAB 的目标是在未知收益环境中，通过逐步积累历史反馈，对各拉杆的价值进行实时估计，并据此动态调整选择策略，使长期累计收益最大化^[16]。与传统的固定轮换或静态概率分配不同，多臂老虎机模型能够基于邻域在近期迭代中的表现动态调整它们在搜索中的参与度。该框架广泛用于算子选择、启发式调度与自适应优化，是处理“经验学习型选择问题”的标准模型。

已有研究表明了多臂老虎机策略在组合优化中具有良好适应性。Junyang 等^[17]在求解混合整数问题的大邻域搜索中引入多臂老虎机控制算子调度，根据搜索过程中的收益自动调节各邻域调用频率，在多个标准测试集上表现均优于固定策略。Lagos 等^[18]将多臂老虎机方法与马尔可夫模型相结合，提出基于概率转移的扰动调度框架，并将其应用于带时间窗的路径规划问题中。算法不仅提升了解的收敛质量，也展现出对扰动算子之间表现差异的自适应识别能力。

在本研究中，每个邻域可被视为“不同收益分布的老虎机”。邻域 N_k 的选

择概率定义为：

$$P(N_k) = \frac{\exp(r_k/\tau)}{\sum_{j=1}^4 \exp(r_j/\tau)} \quad (2)$$

其中 τ 为温度选择参数，控制探索强度，邻域“收益率” r_k 计算公式为：

$$r_k = \frac{\text{succ}_k}{\text{att}_k} \quad (3)$$

算法会为每个邻域计算该邻域的调用次数 att_k 与成功改善次数 succ_k 。每次选择邻域时，算法会将这些收益率作为输入，通过 Softmax 函数构建概率分布，从而以概率方式选择下一个邻域。成功率越高的邻域获得更大概率被调用，而成功率较低的邻域仍保留一定的尝试机会。这种机制与多臂老虎机中的 Boltzmann 探索策略高度一致，使算法能够在利用表现良好的邻域和探索潜在有用但尚未充分尝试的邻域之间形成有效的动态平衡。

此外，为使算法具备更强的适应性和鲁棒性，本研究针对性地设计了几项机制。首先，在邻域的调用次数较少时，我们会人为地给予其较高的初始评分，保证每个邻域在搜索早期都能获得充分尝试，避免被过早淘汰。

$$r_k \leftarrow r_k + c_0 \quad (4)$$

此外，即使某邻域近期未取得成功，算法也会在评分中加入一个基础“探索项”，确保其不会在 Softmax 中彻底失去权重。

$$r_k \leftarrow r_k + c_{exp} \quad (5)$$

当累计尝试次数超过阈值时，算法会对历史统计的占比进行周期性衰减，使邻域选择更关注近期表现，增强适应性。

$$\text{att}_k \leftarrow \frac{1}{2} \text{att}_k, \quad \text{succ}_k \leftarrow \frac{1}{2} \text{succ}_k \quad (6)$$

最后，算法根据每个邻域当前的收益值，采用轮盘赌的策略，以更大的概率选择收益值更大的邻域对解进行操作变化，搜索更优解。

通过以上设计，自适应邻域选择模块能够根据模拟退火过程中各邻域的实际表现自动调整策略：在搜索初期，系统温度较高，重构幅度较大的邻域 N3 以及全局破坏与重构模块更可能带来显著改善，其成功率因此提高，进而获得更高调用概率；而在温度下降、局部搜索更重要的阶段，路径级精细邻域的相对成功率会上升，其在 Softmax 中的贡献自然增强。这种与模拟退火判定机制相耦合的“间

接阶段性切换”，使整体算法能够在大规模搜索空间中实现“早期探索—后期收敛”的动态调控模式。

3.8 重启动机制

在模拟退火框架下，尽管邻域搜索、破坏与重构和温度控制能够在一定程度上减少早熟收敛，但在实际问题中，算法仍可能在某个局部结构附近长时间徘徊，效率有待改善。因此，算法设计了重启动机制，在判定搜索陷入明显停滞，将当前解完全丢弃，并在保持参数设置不变的前提下重新生成一份新的初始解进行搜索。

具体而言，当记录连续无改进次数的计数器达到预设上限时，算法触发重启动操作。首先，将当前解中所有 `nets` 的路径信息、通道使用统计以及拓扑调整记录全部清空，使通道占用、延迟评估等全局状态恢复到未布线的初始状态。随后，将所有 `nets` 的编号顺序进行一次随机打乱，并在这一新的顺序下重新调用初始化过程。由于路径生成过程本身是动态贪心式的，因此改变 `nets` 的处理顺序可以显著改变后续最短路的选择，从而引导算法的初始化落入不同的解域。与此同时，模拟退火的温度也被恢复到初始高温水平，使算法在新的解域中再次拥有较强的接受劣解能力和全局探索能力。

由于在大规模算例中邻域操作的运行成本往往较高，在明显停滞阶段继续进行局部扰动往往难以带来与时间相匹配的收益。重启动机制一方面避免了在原有解附近做大量无效尝试，节省计算时间；另一方面，通过随机化初始化路径构造的起点，能够促使算法探索多个不同的解簇，从而显著提升在复杂搜索空间中找到高质量布线方案的概率。

3.9 模拟退火

在多邻域搜索框架中，邻域扰动会不断对解结构产生变化，但若始终以贪心方式仅接受更优解，则搜索极易在早期陷入局部最优；相反，若扰动过于随意，又会导致解结构大幅震荡，难以收敛。因此，本算法采用模拟退火作为搜索过程的接受准则，在邻域输出候选解后根据当前温度动态决定是否接受该解，从而在探索性与稳定性之间取得平衡。

模拟退火机制的核心在于以温度 T 表示搜索的探索自由度。在搜索初期，温度较高，算法以更大的概率接受劣解，使搜索更容易跳出局部最优；随着搜索过

程进行,温度不断下降,系统逐渐趋于稳定,倾向于在较小范围内作细粒度调整。

具体而言,当某次邻域操作生成一个新解 S' ,算法计算其与当前解 S 在目标函数上的差值 $\Delta = f(S') - f(S)$ 。若 $\Delta < 0$ (即新解更优),则无条件接受;若 $\Delta \geq 0$,则以一定的概率接受新解。

$$P_{accept} = \begin{cases} 1, & \Delta < 0 \\ \exp(-\frac{\Delta}{T}), & \Delta \geq 0, \end{cases} \quad (7)$$

这样的概率接受方式使模拟退火不同于纯粹的随机跳跃:当 Δ 较小或温度较高时,算法仍有一定概率通过接受略差解走出当前局部区域,而当 Δ 较大或温度较低时,接受概率迅速下降,使搜索保持稳定。

在实际实现中,温度采用多次线性降温策略更新:

$$T \leftarrow \alpha \cdot T, \quad 0 < \alpha < 1 \quad (8)$$

其中 α 通常设为一个接近 1 的参数,使温度下降过程较为平滑。当温度 T 降低到预设的最小温度 T_{min} 时,算法终止。

四. 实验测试

本章节对所提出算法的性能进行评估，并展示其在 4 组公开算例上的测试结果。代码使用 C++ 编写，实验均在搭载 AMD EPYC 7H12（2.6 GHz）处理器与 16 GB 内存的服务器上完成，运行环境为 Linux 操作系统。每个算例使用不同的随机数种子，分别进行了 10 次独立重复实验，获得的实验结果如表 4-1 所示。

4.1 实验结果分析

表 4-1 中的有效数据为四列，分别呈现了 10 次独立重复实验中，算法在各算例上取得最优解的最优值、平均值、标准差以及取得最优值的次数。

表4-1 公开算例的计算结果

算例	最优解	平均值	标准差	最优解（次数）
Case01	71.2	71.2	0	10
Case02	384.8	387.6	2.8	5
Case03	235.6	245.8	3	2
Case04	269.2	283.2	7.1	1

由表中数据可以发现，在较小规模的算例 Case01 和 Case02 上，算法展现出较好的效率与鲁棒性，可以较为稳定地使算例的目标函数值降至较低的水平。在规模较大的 Case03 和 Case04 上，算法同样可以有效将目标函数值降低至较低水平，同时保持平均值与最优值处于相同量级，展现出较好的鲁棒性。

4.2 收敛性分析

为验证所提出的多邻域模拟退火算法的收敛特性，我们对 4 组公开算例不同时间运行的计算结果进行了记录，并绘制了算法在搜索过程中的最坏路径延迟变化曲线，如图 4-1 所示。

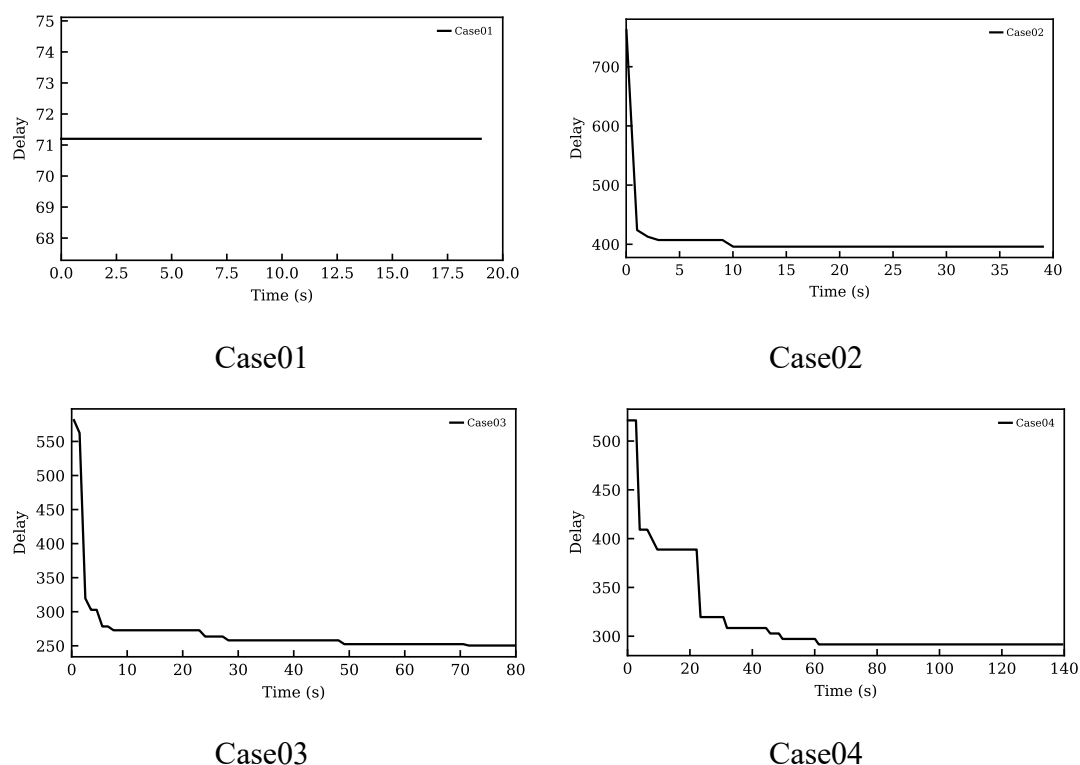


图4-1 各算例收敛曲线

如图所示，除 Case01 规模较小、在初始化阶段即达到稳定延迟外，其余三个算例在搜索过程中均出现了显著的延迟下降，并最终收敛于稳定值，说明算法在实际问题中具备良好的收敛能力。

进一步观察 Case02-Case04 的下降轨迹可发现，它们在搜索初期下降速度较快，而在后期则趋于平缓。这一现象反映出本算法在不同阶段的优化特点。在初期，通过新增通道等拓扑调整操作可迅速缓解关键拥塞，快速降低延迟；而随着可加通道达到上限，算法需依赖路径重构与全局扰动等邻域在固定资源下优化路径结构，表现为延迟下降呈现阶梯式特征。

4.3 消融实验

为进一步验证各邻域模块对整体性能的贡献，本研究对算法中 $N_1 - N_4$ 分别进行了消融实验。以 Case03 与 Case04 为例，逐次禁用每个邻域，并在相同参数与时间设置下记录最大路径延迟随时间的变化曲线，实验结果见图 4-2 与图 4-3 所示。

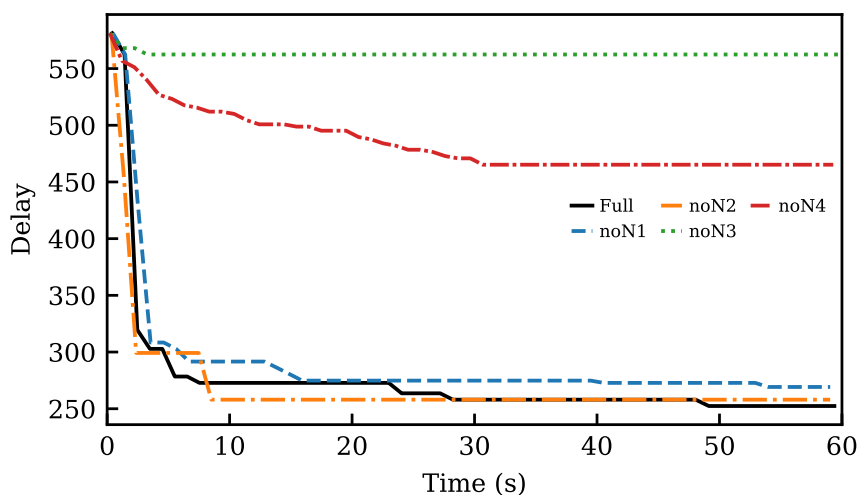


图4-2 case3 消融实验结果图

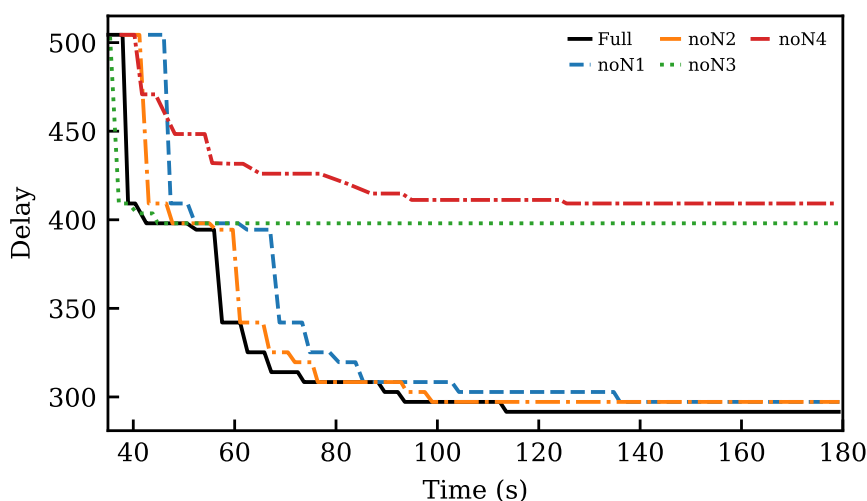


图4-3 case4 消融实验结果图

从结果可以看出，N1-N4 均对算法的性能有一定的提升，其中 N3 与 N4 的作用最为明显。N₃中通道的增加与调整为路径优化提供了关键的资源支持，是搜索前期快速下降的主要推动力。N₄的全局破坏与重构机制则在中后期帮助算法突破结构瓶颈。当其被禁用后，最坏路径延迟长期维持在次优水平，验证了其在打破耦合结构、重组路径方面的关键作用。相较而言，禁用 N₁或 N₂时，算法在后期的局部改进能力有所减弱，更容易在早期陷入停滞，且最终结果不如完整版本，体现出两者在跳出局部最优方面的补充作用。

4.4 自适应邻域选择策略有效性

为验证自适应邻域选择策略的实际效果，本研究设计了对比实验，将原算法中的 Softmax 调度机制替换为等概率随机选择，并对 Case02-Case04 三个算例进

行测试，结果如所示。

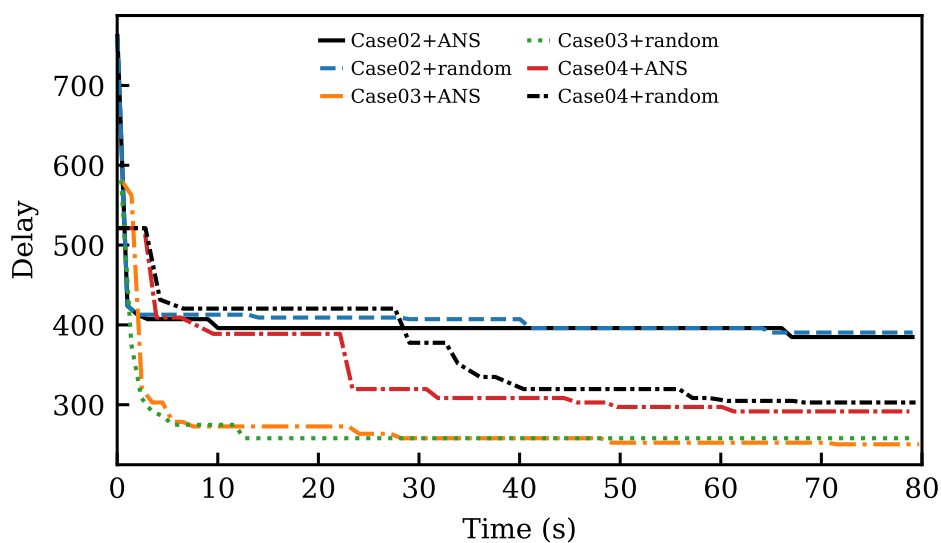


图4-4 自适应邻域选择策略有效性对比图

从图中可以看出，在测试算例中，采用自适应邻域选择机制的算法在最终收敛质量上均优于随机选择版本，且在搜索初期的下降速度更快、后期的优化更稳定。

上述实验结果表明，自适应邻域选择策略能够更有效地调度不同作用的邻域资源，根据各邻域在不同阶段的表现动态调整调用频率，提升整体优化效率，验证了该调度机制在多邻域优化框架下的适用性与实用价值。

五. 总结与展望

5.1 总结

本研究针对多 FPGA 系统级布线中路径选择、通道容量与 TDM 比率三者高度耦合所造成的优化困难,提出了一种基于模拟退火机制的自适应邻域搜索算法。该框架在路径级、拓扑级与全局级三个尺度上设计邻域,实现了路径重构、通道增减与整体结构重构的协同优化;同时,通过多臂老虎机思想构建自适应邻域选择机制,使算法能够在不同搜索阶段自动平衡探索与利用。为进一步增强算法的时间利用效率,框架还引入了基于随机 net 顺序的重启动策略,使搜索能够在长期停滞迅速切换至新的解域。

实验结果显示,所提出的算法在多组公开算例上取得了优异表现,不仅能在严格的通道容量约束与 TDM 比率限制下找到较高质量的布线解,而且在复杂拓扑与大规模实例中也能在较短的时间内取得较好的结果。消融实验进一步验证了各邻域在提升整体解质量中的重要性。同时,对比实验也说明了自适应邻域选择机制相较于随机调度在收敛速度与最终结果上的显著优势。

总体而言,本文所提出的优化框架较传统固定拓扑、单一邻域或静态搜索策略具有更好的适应性和扩展性。其思想对于其他具有强耦合特征的大规模组合优化问题亦具有较高的参考价值。

5.2 未来展望

本研究算法仍可从以下方面进行优化提升:1. 设计快速计算评估方法或次级评价指标,以快速评估新解的目标函数值或使用能反映实际目标函数值水平的次级评价指标指导算法搜索,以提升算法效率,增强算法性能;2. 引入不可行域搜索,适当松弛问题的部分约束,并设计不可行解的修复或惩罚机制,在搜索过程中探索一定的不可行域,并使用修复或惩罚机制获得最终的合法解,以在不可行边界探索更高质量的可行解。

同时,本研究所提算法具有结构简单、量级较轻的特点,无需预先训练和占用大规模的内存,可在较短时间内快速给出可行解。未来研究可继续探索算法的通用性与适配性,为相关实际生产的快速部署与类似问题通用化解决服务。

参考文献

- [1] McMurchie L, Ebeling C. PathFinder: A negotiation-based performance-driven router for FPGAs; proceedings of the Proceedings of the 1995 ACM third international symposium on Field-programmable gate arrays, F, 1995 [C].
- [2] Turki M, Marrakchi Z, Mehrez H, Abid M. Iterative routing algorithm of inter-FPGA signals for multi-FPGA prototyping platform; proceedings of the International Symposium on Applied Reconfigurable Computing, F, 2013 [C]. Springer.
- [3] Tung-Wei L, Wei-Chen T, Yu-Cheng L, Jiang I H-R. Routing Topology and Time-Division Multiplexing Co-Optimization for Multi-FPGA Systems [J]. Proceedings of the 2020 Design Automation Conference, 2020.
- [4] Zou P, Lin Z, Shi X, Wu Y, Chen J, Yu J, Chang Y-W. Time-division multiplexing based system-level FPGA routing for logic verification; proceedings of the 2020 57th ACM/IEEE Design Automation Conference (DAC), F, 2020 [C]. IEEE.
- [5] Khalid M A, Rose J. A novel and efficient routing architecture for multi-FPGA systems [J]. IEEE transactions on very large scale integration (VLSI) systems, 2000, 8(1): 30-39.
- [6] Kirkpatrick S, Gelatt C D, Vecchi M P. Optimization by simulated annealing [J]. Readings in Computer Vision, 1987: 606-615.
- [7] Van P J M L, Aarts E H L. Simulated Annealing: Theory and Applications [J]. DReidel Publishing Company, 1987.
- [8] Vermorel J, Mohri M. Multi-armed bandit algorithms and empirical evaluation; proceedings of the European conference on machine learning, F, 2005 [C]. Springer.
- [9] Benaicha R, Taibi M. Dijkstra algorithm implementation on fpga card for telecom calculations [J]. International Journal of Engineering Sciences and Emerging Technologies, 2013, 4(2): 110-116.
- [10] Blum C. Beam-ACO—hybridizing ant colony optimization with beam search: an application to open shop scheduling [J]. Computers & Operations Research, 2005, 32(6): 1565-1591.
- [11] Lowerre B T. The Harpy speech recognition system [M]. The Harpy speech recognition system, 1976.
- [12] Marchionini G. Exploratory search: from finding to understanding [J]. Communications of the ACM, 2006, 49(4): 41-46.
- [13] Nazari M, Oroojlooy A, Snyder L, Takác M. Reinforcement learning for solving the vehicle routing problem [J]. Advances in neural information processing systems, 2018, 31.
- [14] Choo J, Kwon Y-D, Kim J, Jae J, Hottung A, Tierney K, Gwon Y. Simulation-guided beam search for neural combinatorial optimization [J]. Advances in Neural Information Processing Systems, 2022, 35: 8760-8772.
- [15] Lin F T, Kao C Y, Hsu C C. Applying the genetic approach to simulated

-
- annealing in solving some NP-hard problems [J]. IEEE Transactions on Systems Man & Cybernetics, 1993, 23(6): 1752-1767.
- [16] Kuleshov V, Precup D. Algorithms for multi-armed bandit problems [J]. arXiv preprint arXiv:14026028, 2014.
- [17] Junyang Cai S K, Bistra Dilkina. Balans: Multi-Armed Bandits-based Adaptive Large Neighborhood Search for Mixed-Integer Programming Problems [J]. Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, 2025: 2566-2574.
- [18] Lagos F, Pereira J. Multi-armed bandit-based hyper-heuristics for combinatorial optimization problems [J]. European Journal of Operational Research, 2024, 312(1): 70-91.